

Designing a 5-stage pipeline RISC processor (with hazard detection and data forwarding unit as necessary) that can execute the following instructions:

0000 ADD reg1, reg2, reg3

0004 SLTI reg4, reg1, 9001

0008 LW reg6, 9(reg5)

0012 NAND reg8, reg6, reg7

0016 SUB reg10, reg9, reg8

Initialization Data:

reg2 = 671EB reg3 = 409AF

reg5 = 7 reg7 = F1129

reg9 = CAB77

Instruction Format:

31	28	27	24	23	20	19	16	15	0
OPCODE		Dest Reg (WN)		Source Reg 1 (RN1)		Source Reg 2 (RN2)		Immediate value	

Opcode for each instruction:

Instruction 1: 0000

Instruction 2: 0001

Instruction 3: 0011

Instruction 4: 0111

Instruction 5: 1111

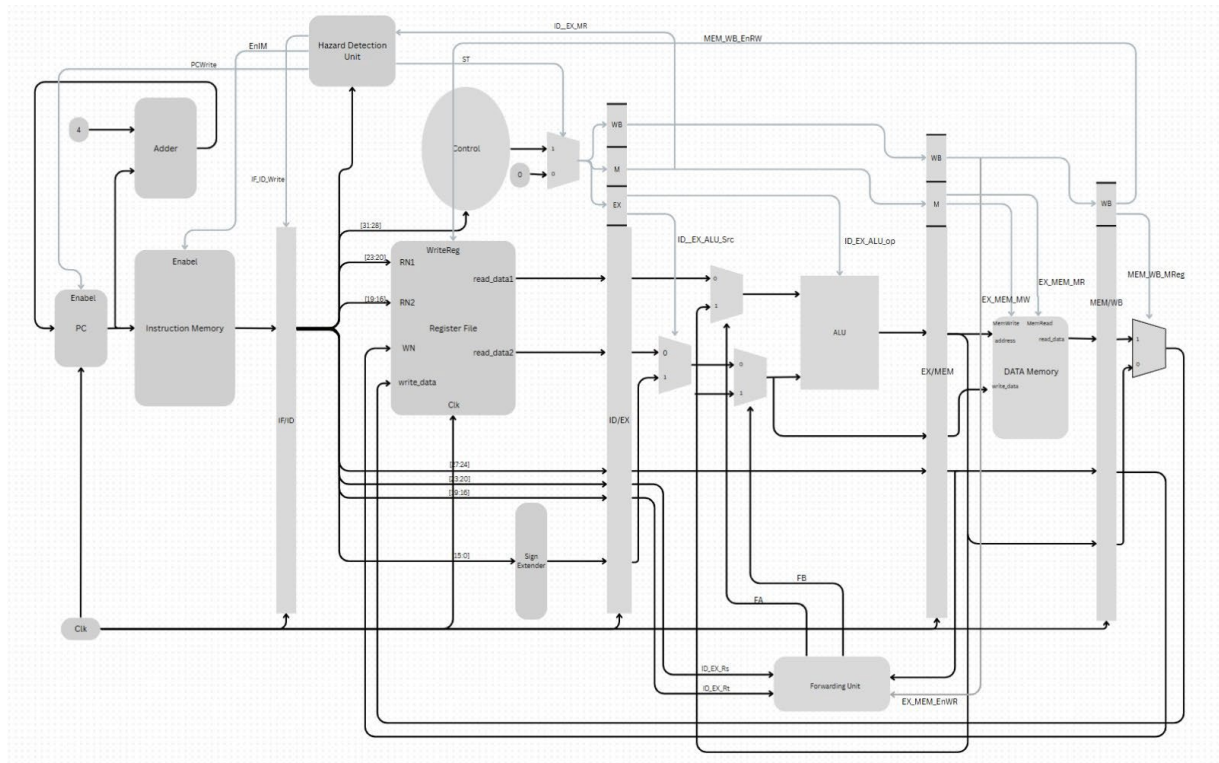
The processor has the following control signals:

- **ALUSrc** - Select the second input of ALU
- **ALUOp (2 bits)** - Control ALU operation
- **MR** - Read data from memory

- **MW** - Write data into memory
- **MReg** - Move data from memory to register
- **EnIM** - Read instruction memory contents
- **EnRW** - Write data into the register file
- **FA** - Forward A mux control (used in data forwarding circuitry)
- **FB** - Forward B mux control (used in data forwarding circuitry)
- **IFIDWrite** - Disable IF/ID change (used in hazard detection circuit)
- **PCWrite** - Disable PC change (used in hazard detection circuit)
- **ST** - Control signal of mux which changes all control signals to zero (used in hazard detection circuit)

Initialize PC with all zeros. The instruction memory is of size 32 bytes, and the processor has 16 registers, numbered from reg0 to reg15, and the registers are 32 bits wide. A read operation from the instruction memory outputs 4 consecutive bytes of information (starting from the byte address provided to the memory) at the positive edge of the clock if the EnIM control signal is high. Assume that the register file has two 32-bit read ports: RD1 and RD2, for data reading and one 32-bit write port: WD, for data writing. At the rising edge of the clock, the read ports RD1 and RD2 output the data from the registers whose addresses are available at RN1 and RN2, respectively. At the falling edge of a clock, data is written via the write port WD to the register file whose address is present at WN if the EnRW signal is true. Design the data memory size as per the requirement. All the data are in hexadecimal format unless specified.

PROJECT: BLOCK DIAGRAM



FUNCTIONS OF ALL THE CORE PROCESSING UNITS:

1. Instruction Memory-

Has all the program instructions stored in contiguous memory locations. It is of size 32 bytes (32 locations of 8 bits or 1 byte each) and four such consecutive locations constitute one instruction of fixed size 32 bits. In our implementation and in general cases, after each interval of 4 bytes starting from **00000000H**, a new instruction starts. The address of the instruction to be executed in a particular cycle comes from the Program Counter (PC) and at every clock cycle (CLK), it outputs the stored instruction at that location. The RESET signal initializes the value of PC, i.e., sets it to **32'd0** while EnIM (Enable Instruction Memory) activates or deactivates the memory based on associated pipeline conditions.

2. Adder-

A specialized arithmetic component that increments the Program Counter by 4 (for word-aligned instructions) to generate the next sequential instruction address. This value feeds back to the PC register for the next instruction fetch cycle, unless a branch or jump instruction alters the control flow.

3. Control Unit –

Decodes and extracts information from the 32-bit instruction to determine its outputs which would determine the actions to be done in the datapath and its components. For different opcode values, it generates the appropriate control signals for the ALU, data memory and register banks and the multiplexors. Specifically, these are ALU operation codes (ALUOPC), memory operation signals (MR, MW), register write signals (WN, WD), and various multiplexor select lines. It also handles condition codes and status flags for conditional execution. Also has an input ***stall_control***, which ensures that a stall operation is conducted when a data hazard occurs and sets all control signals to 0, ensuring no operation.

4. Register Bank-

A bank of general-purpose registers that are used to store temporary data during program execution. 16 registers each of 32 bits size each are present, addressed as ***registers[0]*** to ***registers[15]***. Two ports for reading data are there, while one is available for inputting data (writing) into one of the registers. The ports to be read (source registers) are addressed by ***rs or RN1, instruction [20:23]*** and ***rt or RN2, instruction [16:19]***. The register where data is to be written (destination register) is addressed by ***rd or WN, instruction [24:27]***. An input port ***write_data*** (Write Data, the actual data to be written) brings the data to be put into the WN register. Ports ***read_data1*** and ***read_data2*** take the data read from the registers ***rs*** and ***rd*** respectively as outputs. At the rising edge of the clock, the read ports ***RD1*** and ***RD2*** output the data from the registers whose addresses are available at ***RN1*** and ***RN2***, respectively. At the falling edge of a clock, data is written via the write port ***WD*** to the register file whose address is present at ***WN*** if the ***EnRW*** signal is true.

5. ALU (Arithmetic Logic Unit)-

Performs arithmetic and logical operations based on the **ALU_Op** and **ALU_Src** signals, which in turn depend on the instruction. The inputs A and B come from multiplexors, which select either direct register values, immediate data or forwarded values from later pipeline stages (done by forwarding unit). Generally, operations include ADD, SUB, AND, OR, shifts, etc. In our case, we only make use of the addition, subtraction, nand and compare (logical) functionalities. The ALU outputs the result of the operation, labelled as output **result**, and a zero-flag labelled **zero**, normally used for branch decisions.

6. ALU Control Unit-

Generates specific ALU operation codes based on the **funct** field (6-bit functional field obtained from R-type instructions) and the 2-bit **ALUOp** control signal. This unit allows the ALU block to perform the precise operation required by the current instruction, whether it's an R-type, I-type, or any other instruction format.

7. Hazard Detection Unit-

Checks for instruction dependencies to identify situations where the pipeline must stall. It receives register information (4-bit values) from current and preceding instructions, specifically, **rt** from the EX stage and **rs** and **rt** from the ID stage and detects load-use hazards where an instruction tries to use data that hasn't yet been loaded from memory. When a hazard is detected, it enables (sets as 1) the stall signal (ST) to suspend parts of the pipeline and prevents the PC from incrementing, effectively inserting a bubble (NOP-no operation stage) in the pipeline.

8. Forwarding Unit-

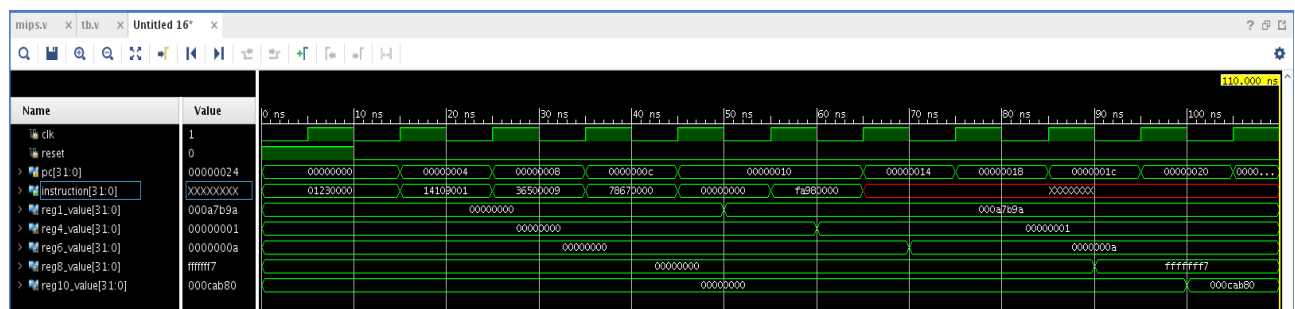
Detects and helps in resolving data hazards caused by instruction and data dependencies in the pipeline. The source register numbers (**rs** and **rt**) of the instruction in the EX stage with the destination registers of instructions in later pipeline stages (EX/MEM Register **Rd** and MEM/WB Register **Rd**) are compared and if a match is found, indicating that a previous instruction is producing a result needed by the current instruction, the forwarding unit controls the multiplexers (**forward_muxa** and **forward_muxb**) with select lines (outputs of forwarding unit) **FA** and

FB to route the data directly from the pipeline registers rather than waiting for it to be written back to the register file. This unit allows execution to proceed without needless stalls by receiving inputs from numerous pipeline registers and sending control signals to the ALU input multiplexers. It basically helps in bypassing unnecessary stages in the datapath and forwarding data directly to the required destinations.

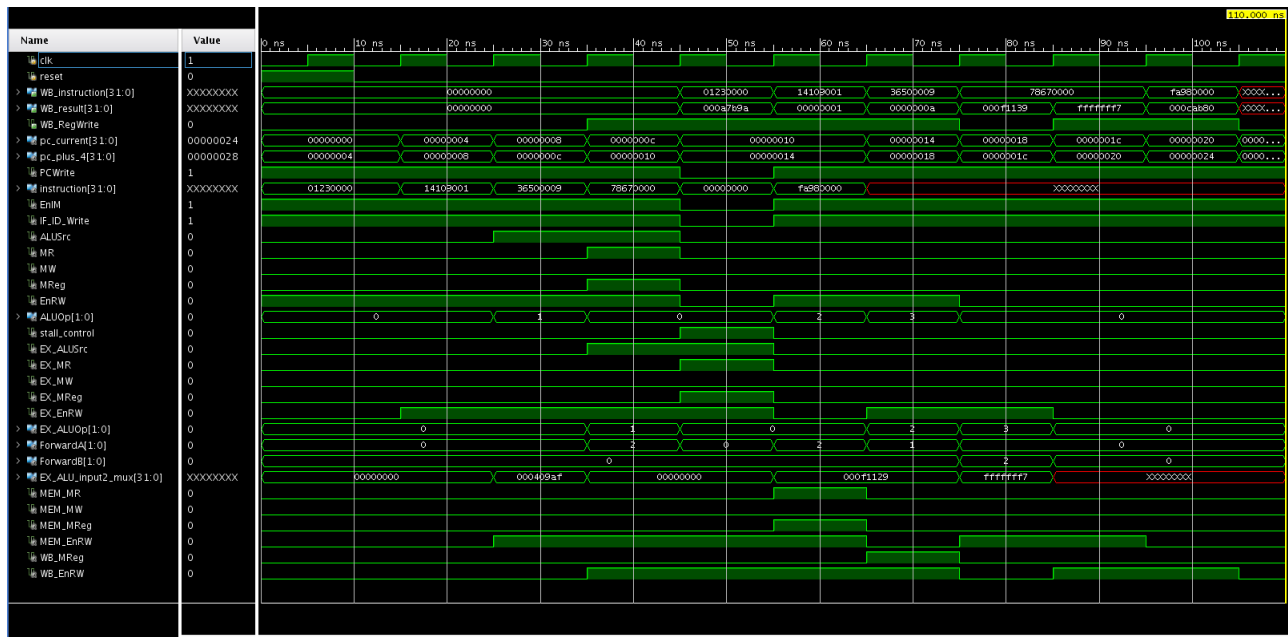
9. Data Memory-

Stores program data (as opposed to instructions) in addressable memory locations, where 24 locations of 32 bits each are present. Address input comes from **address** (ALU Result of LW instruction), and data input/output happens through **write_data** and **read_Data** ports respectively. Memory access is controlled by **MR** (Memory Read) and **MW** (Memory Write) signals, according to whether data is **read** from memory (LW instruction) or **written** into memory (SW instruction). The memory operations are synchronized with the clock (CLK).

Output Waveforms



Control Signals



IF/ID Pipelined Register – 32 bits

ID/EX Pipelined Register – 115 bits

EX/MEM Pipelined Register – 72 bits

MEM/WB Pipelined Register – 70 bits